

Problem Analysis using OOP Four Pillars with JAVA

Problem Statement :

1. University Management System

Problem Statement

Design a system for a university where students and teachers are managed. Students can enroll in courses, and teachers can teach multiple courses.

Requirements

- Student and Teacher have common attributes (name, id)
- A student can enroll in multiple courses
- A teacher can teach multiple courses
- Each course has a title and code

OOP Usage

- Encapsulation → private data (id, name)
- Inheritance → Person → Student, Teacher
- Polymorphism → displayInfo() different for Student & Teacher
- Abstraction → Person class or interface

UML Idea

- Person (abstract)
 - Student
 - Teacher
- Course
- Association between Student ↔ Course, Teacher ↔ Course

Coding Solution :

```
import java.util.ArrayList;

import java.util.List;

// ===== ABSTRACTION + ENCAPSULATION =====

abstract class Person {

    private int id;

    private String name;

    public Person(int id, String name) {

        this.id = id;

        this.name = name;

    }

    // getters (Encapsulation)

    public int getId() {

        return id;

    }

    public String getName() {

        return name;

    }

    // abstract method (Polymorphism base)

    abstract void displayInfo();

}
```

```
// ===== COURSE CLASS =====  
  
class Course {  
  
    private String courseCode;  
  
    private String title;  
  
  
    public Course(String courseCode, String title) {  
  
        this.courseCode = courseCode;  
  
        this.title = title;  
  
    }  
  
  
    public String getCourseCode() {  
  
        return courseCode;  
  
    }  
  
  
    public String getTitle() {  
  
        return title;  
  
    }  
  
  
    public void showCourse() {  
  
        System.out.println(courseCode + " - " + title);  
  
    }  
}
```

```
// ===== STUDENT (INHERITANCE + POLYMORPHISM) =====

class Student extends Person {

    private List<Course> courses = new ArrayList<>();

    public Student(int id, String name) {

        super(id, name);

    }

    public void enrollCourse(Course c) {

        courses.add(c);

    }

    @Override
    void displayInfo() {

        System.out.println("Student ID: " + getId());

        System.out.println("Student Name: " + getName());

        System.out.println("Enrolled Courses:");

        for (Course c : courses) {

            c.showCourse();

        }

    }

}
```

```
// ===== TEACHER (INHERITANCE + POLYMORPHISM) =====

class Teacher extends Person {

    private List<Course> courses = new ArrayList<>();

    public Teacher(int id, String name) {

        super(id, name);

    }

    public void assignCourse(Course c) {

        courses.add(c);

    }

    @Override

    void displayInfo() {

        System.out.println("Teacher ID: " + getId());

        System.out.println("Teacher Name: " + getName());

        System.out.println("Teaching Courses:");

        for (Course c : courses) {

            c.showCourse();

        }

    }

}
```

```
// ===== MAIN CLASS =====

public class Main {

    public static void main(String[] args) {

        Course c1 = new Course("CSE101", "Programming Basics");

        Course c2 = new Course("CSE102", "Data Structures");

        Student s1 = new Student(1, "Khalid");

        s1.enrollCourse(c1);

        s1.enrollCourse(c2);

        Teacher t1 = new Teacher(100, "Dr. Ahmed");

        t1.assignCourse(c1);

        System.out.println("===== STUDENT INFO =====");

        s1.displayInfo();

        System.out.println("\n===== TEACHER INFO =====");

        t1.displayInfo();

    }

}
```